Assignment - 16.3

Name : Manupati Eshwar
H.No: 2303A51495
Batch - 08

Lab 16 – Database Design and Queries: Schema Design and SQL Generation

Task-1:Hospital Management Database

Promt:

Act as a Database Architect. Design a normalized (3NF) SQL schema for a Hospital Management System. Include three tables: Doctors, Patients, and Appointments. Define clear constraints such as Primary Keys, Foreign Keys, unique contacts for patients, and valid date formats. After creating the schema, generate SQL queries to: 1) List all appointments for a specific DoctorID, 2) Retrieve complete patient history using a PatientID, and 3) Count the total number of unique patients treated by each doctor.

```
    ->     appointment_date DATE NOT NULL,
    ->     diagnosis TEXT,
    ->     FOREIGN KEY (doctor_id) REFERENCES Doctors(doctor_id),
    ->     FOREIGN KEY (patient_id) REFERENCES Patients(patient_id)
    -> );
ERROR 1046 (3D000): No database selected
mysql> CREATE DATABASE HospitalDB;
Query OK, 1 row affected (0.003 sec)

mysql> USE HospitalDB;
Database changed
mysql> CREATE TABLE Doctors (
    ->     doctor_id INT PRIMARY KEY,
    ->     name VARCHAR(100) NOT NULL,
    ->     specialization VARCHAR(100),
    ->     phone VARCHAR(15) UNIQUE
    -> );
Query OK, 0 rows affected (0.008 sec)

mysql>
mysql> CREATE TABLE Patients (
    ->     patient_id INT PRIMARY KEY,
    ->     name VARCHAR(100) NOT NULL,
    ->     age INT CHECK (age > 0),
    ->     phone VARCHAR(15) UNIQUE
    -> );
Query OK, 0 rows affected (0.004 sec)

mysql>
mysql> CREATE TABLE Appointments (
    ->     appointment_id INT PRIMARY KEY,
    ->     doctor_id INT,
    ->     patient_id INT,
    ->     appointment_date DATE NOT NULL,
    ->     diagnosis TEXT,
    ->     FOREIGN KEY (doctor_id) REFERENCES Doctors(doctor_id),
    ->     FOREIGN KEY (patient_id) REFERENCES Patients(patient_id)
    -> );
Query OK, 0 rows affected (0.011 sec)

mysql>
```

```
mysql> SELECT a.*, p.name AS patient_name
    -> FROM Appointments a
    -> JOIN Patients p ON a.patient_id = p.patient_id
    -> WHERE a.doctor_id = 101;
+----------------+-----------+------------+------------------+---------------+--------------+
| appointment_id | doctor_id | patient_id | appointment_date | diagnosis     | patient_name |
+----------------+-----------+------------+------------------+---------------+--------------+
|              1 |       101 |        201 | 2026-03-01       | Heart Checkup | Alice        |
|              2 |       101 |        202 | 2026-03-05       | BP Issue      | Bob          |
+----------------+-----------+------------+------------------+---------------+--------------+
2 rows in set (0.001 sec)
```

```
mysql> SELECT a.*, d.name AS doctor_name
    -> FROM Appointments a
    -> JOIN Doctors d ON a.doctor_id = d.doctor_id
    -> WHERE a.patient_id = 201;
+----------------+-----------+------------+------------------+---------------+-------------+
| appointment_id | doctor_id | patient_id | appointment_date | diagnosis     | doctor_name |
+----------------+-----------+------------+------------------+---------------+-------------+
|              1 |       101 |        201 | 2026-03-01       | Heart Checkup | Dr. Smith   |
|              3 |       102 |        201 | 2026-03-10       | Migraine      | Dr. John    |
+----------------+-----------+------------+------------------+---------------+-------------+
2 rows in set (0.001 sec)
```

```
mysql> SELECT d.name, COUNT(DISTINCT a.patient_id) AS total_patients
    -> FROM Doctors d
    -> LEFT JOIN Appointments a ON d.doctor_id = a.doctor_id
    -> GROUP BY d.name;
+-----------+----------------+
| name      | total_patients |
+-----------+----------------+
| Dr. John  |              1 |
| Dr. Smith |              2 |
+-----------+----------------+
2 rows in set (0.001 sec)
```

Task 2 – Real-Time Application: Online Shopping Database

Promt:

Design a database schema for an E-commerce platform with four tables: Users, Products, Orders, and OrderDetails. Ensure the relationship between Orders and OrderDetails is optimized for 3rd Normal Form to handle multiple items per order. Write SQL queries to: 1) Retrieve all order history for a specific user, 2) Identify the top-selling product by quantity, and 3) Calculate the total revenue generated for the current month. Additionally, suggest how to optimize these queries for a high-traffic platform

```
mysql> USE Shopping;
Database changed
mysql> CREATE TABLE Users (
    ->     user_id INT PRIMARY KEY,
    ->     name VARCHAR(100),
    ->     email VARCHAR(100) UNIQUE
    -> );
Query OK, 0 rows affected (0.006 sec)

mysql>
mysql> CREATE TABLE Products (
    ->     product_id INT PRIMARY KEY,
    ->     name VARCHAR(100),
    ->     price DECIMAL(10,2),
    ->     stock INT
    -> );
Query OK, 0 rows affected (0.004 sec)

mysql>
mysql> CREATE TABLE Orders (
    ->     order_id INT PRIMARY KEY,
    ->     user_id INT,
    ->     order_date DATE,
    ->     FOREIGN KEY (user_id) REFERENCES Users(user_id)
    -> );
Query OK, 0 rows affected (0.005 sec)

mysql>
mysql> CREATE TABLE OrderDetails (
    ->     order_detail_id INT PRIMARY KEY,
    ->     order_id INT,
    ->     product_id INT,
    ->     quantity INT CHECK (quantity > 0),
    ->     FOREIGN KEY (order_id) REFERENCES Orders(order_id),
    ->     FOREIGN KEY (product_id) REFERENCES Products(product_id)
    -> );
Query OK, 0 rows affected (0.008 sec)
```

```
Query OK, 1 row affected (0.001 sec)

mysql> SELECT o.order_id, o.order_date, p.name, od.quantity
    -> FROM Orders o
    -> JOIN OrderDetails od ON o.order_id = od.order_id
    -> JOIN Products p ON od.product_id = p.product_id
    -> WHERE o.user_id = 1;
+----------+------------+--------+----------+
| order_id | order_date | name   | quantity |
+----------+------------+--------+----------+
|     1001 | 2026-03-10 | Laptop |        1 |
|     1001 | 2026-03-10 | Phone  |        2 |
+----------+------------+--------+----------+
2 rows in set (0.001 sec)
```

```
ion for the right syntax to' at line 1
mysql> SELECT p.name, SUM(od.quantity) AS total_sold
    -> FROM OrderDetails od
    -> JOIN Products p ON od.product_id = p.product_id
    -> GROUP BY p.name
    -> ORDER BY total_sold DESC
    -> LIMIT 1;
+-------+------------+
| name  | total_sold |
+-------+------------+
| Phone |          5 |
+-------+------------+
1 row in set (0.001 sec)

mysql>
```

```
mysql> SELECT SUM(p.price * od.quantity) AS total_revenue
    -> FROM Orders o
    -> JOIN OrderDetails od ON o.order_id = od.order_id
    -> JOIN Products p ON od.product_id = p.product_id
    -> WHERE EXTRACT(MONTH FROM o.order_date) = 3;
+---------------+
| total_revenue |
+---------------+
|     150000.00 |
+---------------+
1 row in set (0.001 sec)

mysql>
```

Task 3 – Library Database:

Promt:

Create a SQL schema for a Library Management System including Books, Members, and Loans. Suggest an indexing strategy to ensure that searching for issued books and member history remains fast as the database grows. Generate queries to: 1) List all books that are currently issued (not yet returned), 2) Identify overdue books that were loaned more than 30 days ago, and 3) Calculate the total number of books currently borrowed by each member using a LEFT JOIN.

```
mysql> CREATE DATABASE `Library`;
Query OK, 1 row affected (0.002 sec)

mysql> USE `Library`;
Database changed
mysql> CREATE TABLE Books (
    ->     book_id INT PRIMARY KEY,
    ->     title VARCHAR(200),
    ->     author VARCHAR(100)
    -> );
Query OK, 0 rows affected (0.006 sec)

mysql>
mysql> CREATE TABLE Members (
    ->     member_id INT PRIMARY KEY,
    ->     name VARCHAR(100)
    -> );
Query OK, 0 rows affected (0.004 sec)

mysql>
mysql> CREATE TABLE Loans (
    ->     loan_id INT PRIMARY KEY,
    ->     book_id INT,
    ->     member_id INT,
    ->     loan_date DATE,
    ->     return_date DATE,
    ->     FOREIGN KEY (book_id) REFERENCES Books(book_id),
    ->     FOREIGN KEY (member_id) REFERENCES Members(member_id)
    -> );
Query OK, 0 rows affected (0.008 sec)

mysql>
```

```
mysql> SELECT b.title, m.name AS member_name
    -> FROM Loans l
    -> JOIN Books b ON l.book_id = b.book_id
    -> JOIN Members m ON l.member_id = m.member_id
    -> WHERE l.return_date IS NULL;
+-----------------+-------------+
| title           | member_name |
+-----------------+-------------+
| DBMS Basics     | Anu         |
| Data Structures | Raj         |
+-----------------+-------------+
2 rows in set (0.001 sec)

mysql>
```

```
mysql> SELECT b.title, m.name, l.loan_date
    -> FROM Loans l
    -> JOIN Books b ON l.book_id = b.book_id
    -> JOIN Members m ON l.member_id = m.member_id
    -> WHERE l.return_date IS NULL
    -> AND l.loan_date < CURDATE() - INTERVAL 30 DAY;
+-------------+------+------------+
| title       | name | loan_date  |
+-------------+------+------------+
| DBMS Basics | Anu  | 2026-02-14 |
+-------------+------+------------+
1 row in set (0.001 sec)

mysql>
```

```
mysql> SELECT m.name, COUNT(l.book_id) AS total_books
    -> FROM Members m
    -> LEFT JOIN Loans l ON m.member_id = l.member_id
    -> GROUP BY m.name;
+------+-------------+
| name | total_books |
+------+-------------+
| Anu  |           1 |
| Raj  |           2 |
+------+-------------+
2 rows in set (0.001 sec)

mysql>
```

Task 4 :

Promt:

Analyze the following SQL query for efficiency: `SELECT * FROM Books WHERE author_id IN (SELECT author_id FROM Authors WHERE country='UK');`. Explain why a subquery might be inefficient for large datasets and provide an optimized version using an INNER JOIN. Explain how the database engine handles the JOIN differently than the subquery.

Original Query:

```
SELECT *
FROM Books
WHERE author_id IN (
   SELECT author_id FROM Authors WHERE country = 'UK'
);
```

Optimized Query

```
SELECT b.*
FROM Books b
JOIN Authors a ON b.author_id = a.author_id
WHERE a.country = ‘UK';
```

Task 5: University Course Registration

Promt:

Design a university course registration system schema for SR University. The tables should include Students, Courses, Faculty, and Registrations. Define the relationships: students can enroll in multiple courses, and each course is assigned to one faculty member. Write SQL queries to: 1) List all student names enrolled in a specific course title, 2) Identify faculty members who are teaching more than two courses using the HAVING clause, and 3) List all courses ranked by the highest number of student registrations.

```
mysql> CREATE DATABASE `University`;
Query OK, 1 row affected (0.002 sec)

mysql> USE `Univeristy`;
ERROR 1049 (42000): Unknown database 'univeristy'
mysql> USE `University`;
Database changed
mysql> CREATE TABLE Students (
    ->     student_id INT PRIMARY KEY,
    ->     name VARCHAR(100)
    -> );
Query OK, 0 rows affected (0.004 sec)

mysql>
mysql> CREATE TABLE Faculty (
    ->     faculty_id INT PRIMARY KEY,
    ->     name VARCHAR(100)
    -> );
Query OK, 0 rows affected (0.003 sec)

mysql>
mysql> CREATE TABLE Courses (
    ->     course_id INT PRIMARY KEY,
    ->     course_name VARCHAR(100),
    ->     faculty_id INT,
    ->     FOREIGN KEY (faculty_id) REFERENCES Faculty(faculty_id)
    -> );
Query OK, 0 rows affected (0.005 sec)

mysql>
mysql> CREATE TABLE Registrations (
    ->     reg_id INT PRIMARY KEY,
    ->     student_id INT,
    ->     course_id INT,
    ->     FOREIGN KEY (student_id) REFERENCES Students(student_id),
    ->     FOREIGN KEY (course_id) REFERENCES Courses(course_id)
    -> );
Query OK, 0 rows affected (0.007 sec)

mysql>
```

```
mysql> SELECT s.name
    -> FROM Registrations r
    -> JOIN Students s ON r.student_id = s.student_id
    -> WHERE r.course_id = 101;
+--------+
| name   |
+--------+
| Eshwar |
| Kiran  |
+--------+
2 rows in set (0.001 sec)

mysql>
```

```
mysql> SELECT f.name, COUNT(c.course_id) AS total_courses
    -> FROM Faculty f
    -> JOIN Courses c ON f.faculty_id = c.faculty_id
    -> GROUP BY f.name
    -> HAVING COUNT(c.course_id) > 2;
+-----------+---------------+
| name      | total_courses |
+-----------+---------------+
| Prof. Rao |             3 |
+-----------+---------------+
1 row in set (0.001 sec)

mysql>
```

```
mysql> SELECT c.course_name, COUNT(r.student_id) AS total_students
    -> FROM Courses c
    -> JOIN Registrations r ON c.course_id = r.course_id
    -> GROUP BY c.course_name
    -> ORDER BY total_students DESC
    -> LIMIT 1;
+-------------+----------------+
| course_name | total_students |
+-------------+----------------+
| DBMS        |              2 |
+-------------+----------------+
1 row in set (0.001 sec)

mysql>
```